



our shielding . Your smart contracts, our shielding . Your smart c



shieldify



Yeet **NFT Vesting** SECURITY REVIEW

Date: 20 January 2025

CONTENTS

1. About Shieldify Security	3
2. Disclaimer	3
3. About Yeet NFT Vesting	3
4. Risk classification	4
4.1 Impact	3
4.2 Likelihood	4
5. Security Review Summary	4
5.1 Protocol Summary	4
5.2 Scope	5
6. Findings Summary	4
7. Findings	5

1. About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and has secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Rust, Go, Vyper, Move and Cairo.

Learn more about us at shieldify.org.

2. Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3. About Yeet - NFT Vesting

Yeet It is a fully on-chain game with no dominant game theoretic strategy to win. Players can make money in multiple ways.

The game is played by depositing **\$BERA** into the pool, and the winner is rewarded the pool of assets once the finish condition is met. The game also has simple token economics attached to it where we will be giving out daily rewards in the form of **\$YEET** tokens to players who deposit assets into the pool during that period. The rewards are distributed each epoch (24 hours) and are done so pro-rata to each player according to how much **\$BERA** they deposited in the epoch that has just elapsed.

Anyone can also stake their **\$YEET** tokens to earn a share of the assets that are deposited into the pool by players.

4. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4.1 Impact

- **High** - results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** - results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** - losses will be limited but bearable - and covers vectors similar to griefing attacks that can be easily repaired

4.2 Likelihood

- **High** – almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** – still relatively likely, although only conditionally possible
- **Low** – requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5. Security Review Summary

The security review lasted 2 days with a total of 64 hours dedicated by 4 researchers from the Shieldify team.

Overall, the code is well-written. The audit report identified one High-severity issue related to wrong calculation leading to stuck funds, three Medium-severity issues regarding blacklisting logic and safe transfers, along with a single low-severity recommendation.

The Yeet team has been very responsive to the Shieldify research team's inquiries, demonstrating a strong commitment and dedication to the protocol's development. Their proactive security approach is evident, as they have conducted multiple security reviews with various companies, showcasing their thoroughness and dedication to securing their platform.

5.1 Protocol Summary

Project Name	Yeet - NFT Vesting
Repository	nft-vesting
Type of Project	NFT Vesting
Audit Timeline	2 days
Review Commit Hash	35024bf65b5337ee5660ed6b3257d66b
Fixes Review Commit Hash	cb0a1cd4de115df347c40f5086502af8

5.2 Scope

The following smart contracts were in the scope of the security review:

File	nSLOC
NFTVesting.sol	99
Total	99

6. Findings Summary

The following number of issues have been identified, sorted by their severity:

- **Critical/High** issues: **1**
- **Medium** issues: **3**
- **Low** issues: **1**

ID	Title	Severity	Status
[H-01]	Tokens Would Stuck Forever Inside The <code>NFTVesting.sol</code> Contract	High	Fixed
[M-01]	Missing Setter Functions for Modifying <code>Blacklist</code> and <code>Allowlist</code> After Contract Deployment	Medium	Acknowledged
[M-02]	Possible Loss of Funds, Transfer Functions Can Silently Fail	Medium	Fixed
[M-03]	USDC Blacklisted Users Unable to Claim Tokens, Resulting in Stuck Funds	Medium	Acknowledged
[L-01]	Several Variables Should be Marked as <code>immutable</code>	Low	Fixed

7. Findings

[H-01] Tokens Would Stuck Forever Inside The `NFTVesting.sol` Contract

Severity

High Risk

Description

In the `NFTVesting` contract, the total token amount (`_tokenAmount`) is evenly distributed among all NFTs (`_nftAmount`), resulting in a fixed allocation per NFT (`tokenAmountPerNft`). For instance, if `_tokenAmount` is set to `100e18` and `_nftAmount` is `100`, each NFT receives an allocation of `1e18` tokens.

However, a key condition in the claim function prevents blacklisted NFTs from claiming their allocated tokens. The function includes the following check:

```
if (blacklistEnabled && blacklistedTokenIds[tokenId]) {
    revert BlacklistedTokenId(tokenId);
}
```

This ensures that tokens allocated to blacklisted NFTs are never claimed.

Importantly, the contract does not include any mechanism to withdraw, reallocate, or recover these unclaimed tokens. As a result, tokens corresponding to blacklisted NFTs remain permanently locked within the contract, rendering them inaccessible and effectively removing them from circulation.

Location of Affected Code

File: `NFTVesting.sol`

```
if (blacklistEnabled && blacklistedTokenIds[tokenId]) {
    revert BlacklistedTokenId(tokenId);
}
```


Impact

Tokens corresponding to blacklisted NFTs are effectively lost and cannot be accessed or redistributed which causes a loss of the protocol owner's fund.

Proof of Concept

- Initial Token Distribution:
 - `_tokenAmount = 100e18`
 - `_nftAmount = 100`
 - `tokenAmountPerNft = 1e18`
- Blacklist Setup:
 - `blacklistedTokenIds = [1, 2, ..., 10]` (10 NFTs are blacklisted).
 - These NFTs collectively account for $10 * 1e18 = 10e18$ tokens.
- Claims by Eligible NFTs:
 - 90 NFTs claim their allocated $1e18$ tokens each.
 - Total claimed = $90 * 1e18 = 90e18$.
- Tokens Stuck in Contract:
 - The $10e18$ tokens allocated to the blacklisted NFTs remain unclaimed and permanently locked.

Recommendation

Consider recalculating the `tokenAmountPerNft` to exclude blacklisted NFTs from the distribution pool: `uint256 adjustedTokenAmountPerNft = tokenAmount / (nftAmount - blacklistedCount);`

Team Response

Fixed.

[M-01] Missing Setter Functions for Modifying `Blacklist` and `Allowlist` After Contract Deployment

Severity

Medium Risk

Description

The current implementation of the NFTVesting contract does not include setter functions for modifying the blacklist (`blacklistedTokenIds`) and the allowlist (`allowedTokenIds`) after deployment. This design creates an unintended limitation, as the ability to update these lists may be necessary in the future.

Once the blacklist and allowlist are set during deployment, their contents become immutable. This means that no changes can be made to these lists, even if circumstances require adjustments.

For example, there might be situations where certain NFTs need to be blacklisted or removed from the blacklist. Similarly, there could be a need to add or remove NFTs from the allowlist to accommodate changing requirements. The lack of functionality to address such scenarios reduces the flexibility and adaptability of the contract.

Location of Affected Code

File: [NFTVesting.sol](#)

```
constructor(
    IERC20 _token,
    uint256 _tokenAmount,
    uint256 _nftAmount,
    uint256 _startTime,
    uint256 _vestingPeriod,
    uint256[] memory _blacklistedTokenIds,
    uint256[] memory _allowedTokenIds
) {
    // code

    // Set up blacklist
    if (_blacklistedTokenIds.length != 0) {
        blacklistEnabled = true;
        for (uint256 i = 0; i < _blacklistedTokenIds.length; i++) {
@>    blacklistedTokenIds[_blacklistedTokenIds[i]] = true;
        }
    }

    // Set up allowed list
    if (_allowedTokenIds.length != 0) {
        allowedListEnabled = true;
        for (uint256 i = 0; i < _allowedTokenIds.length; i++) {
@>    allowedTokenIds[_allowedTokenIds[i]] = true;
        }
    }

    // code
}
```

Impact

- Since the contract does not provide setter functions to modify the blacklist (`blacklistedTokenIds`) and the allowlist (`allowedTokenIds`) after deployment, once these lists are set, they become immutable.
- The inability to modify the blacklist or allowlist prevents the contract from adapting to future needs. If a particular NFT needs to be blacklisted or added to the allowlist later on, the contract would need to be redeployed, which is costly and impractical.

Recommendation

Consider introducing the following setter functions:

- `addToBlacklist(uint256 tokenId)` : To add a token ID to the blacklist.
- `removeFromBlacklist(uint256 tokenId)` : To remove a token ID from the blacklist.
- `addToAllowlist(uint256 tokenId)` : To add a token ID to the allowlist.
- `removeFromAllowlist(uint256 tokenId)` : To remove a token ID from the allowlist.

Team Response

Acknowledged.

[M-02] Possible Loss of Funds, Transfer Functions Can Silently Fail

Severity

Medium Risk

Description

The `NFTVesting::claim()` function does not check the return value of these calls. Tokens that return false rather than revert to indicate failed transfers may silently fail rather than revert as expected.

Since the IERC20 interface requires a boolean return value, attempting to transfer ERC20s with [Missing return values](#) will revert. This means it cannot support a number of popular ERC20s, including USDT, BNB and many others.

Location of Affected Code

File: [NFTVesting.sol](#)

```
function claim(uint256 tokenId) public {
    if (blacklistEnabled && blacklistedTokenIds[tokenId]) {
        revert BlacklistedTokenId(tokenId);
    }

    if (allowedListEnabled && !allowedTokenIds[tokenId]) {
        revert NotAllowedTokenId(tokenId);
    }

    require(nftContract.ownerOf(tokenId) == msg.sender, "NFTVesting: not
        the owner of the NFT");

    require(block.timestamp >= startTime, "NFTVesting: vesting period has
        not started yet");
}
```



```

uint256 claimable = claimable(tokenId);
require(claimable != 0, "Nothing to claim");
require(token.balanceOf(address(this)) >= claimable, "Not enough tokens
    in contract");

claimed[tokenId] += claimable;

@>token.transfer(msg.sender, claimable);
    emit Claimed(tokenId, claimable);
}

```

Impact

Tokens that don't actually perform the transfer and return false are still counted as a correct transfer and tokens that don't correctly implement the latest EIP20 spec will be unusable in the protocol as they revert the transaction because of the missing return value. This will lead to loss of user funds as they haven't actually claimed the token but the mapping of claimed[tokenId] has been updated.

Proof of Concept

- Some tokens do not return a bool (e.g. USDT, BNB, OMG) on ERC20 methods. [See](#) here for a comprehensive list:
- [ReturnsFalse.sol](#)
- [MissingReturns.sol](#)

Recommendation

Consider using a safe transfer library like OpenZeppelin [SafeERC20](#) to ensure consistent handling of ERC20 return values and abstract over inconsistent ERC20 implementations.

Team Response

Fixed.

[M-03] USDC Blacklisted Users Unable to Claim Tokens, Resulting in Stuck Funds

Severity

Medium Risk

Description

If the token used in the [NFTVesting](#) contract is USDC (or a similar centralized token with blacklist-ing capabilities), users blacklisted by the token issuer, such as Circle, cannot claim their tokens.

The unclaimed tokens remain stuck in the contract indefinitely, causing potential losses for both the user and the protocol.

For example, if User A's wallet is blacklisted by Circle, they won't be able to claim their tokens, even though they own an eligible NFT. This issue highlights the limitations of using centralized tokens in the [NFTVesting](#) contract.

- The blacklisted user calls the `claim()` function to retrieve their tokens:

```
token.transfer(msg.sender, claimable);
```

USDC enforces blacklist checks during transfers, causing claims by blacklisted users to fail. Tokens allocated to blacklisted NFTs remain permanently stuck in the contract.

Location of Affected Code

File: [NFTVesting.sol](#)

```
function claim(uint256 tokenId) public {
    // code

    uint256 claimable = claimable(tokenId);
    require(claimable != 0, "Nothing to claim");
    require(token.balanceOf(address(this)) >= claimable, "Not enough tokens
        in contract");

    claimed[tokenId] += claimable;

    @>token.transfer(msg.sender, claimable);
    emit Claimed(tokenId, claimable);
}
```

Impact

- Unclaimed USDC tokens are permanently stuck in the contract.
- The protocol cannot recover or reallocate these tokens to other users or to itself without implementing a recovery mechanism.

Recommendation

Consider allowing the contract owner to withdraw tokens allocated to blacklisted user:

```
function recoverUnclaimedTokens() external onlyOwner {
    uint256 unclaimedTokens = token.balanceOf(address(this));
    token.transfer(owner, unclaimedTokens);
}
```

Team Response

Acknowledged.

[L-01] Several Variables Should be Marked as `immutable`

Severity

Low Risk

Description

In the `NFTVesting.sol` contract, several state variables are not marked as `immutable`, but they should be. Specifically, the following variables should be immutable because they are only set once during the contract initialization and should not change afterwards:

- `NFTVesting.allowedListEnabled`
- `NFTVesting.blacklistEnabled`
- `NFTVesting.nfAmount`
- `NFTVesting.startTime`
- `NFTVesting.token`
- `NFTVesting.tokenAmount`
- `NFTVesting.tokenAmountPerNft`
- `NFTVesting.vestingPeriod`

Recommendation

According to the current logic of the contract as these variables are not changed after deployment each of the above variables should be updated to be `immutable`. This change ensures that these variables are only set once during contract initialization and can't be altered after the contract is deployed

Team Response

Fixed.

our shielding . Your smart contracts, our shielding . Your smart c



shieldify



Thank you!

